

FORGE: Forming Semantic Identifiers for Generative Retrieval in Industrial Datasets

Kairui Fu^{z*} Tao Zhang^{a*} Shuwen Xiao^{a*} Ziyang Wang^a
 Xinming Zhang^a Chenchi Zhang^a Yuliang Yan^{a†} Junjun Zheng^a
 Yu Li^a Zhihong Chen^a Jian Wu^a Xiangheng Kong^a
 Shengyu Zhang^z Kun Kuang^{z†} Yuning Jiang^a Bo Zheng^a
^z Zhejiang University ^a Alibaba Group
 Project Page: <https://huggingface.co/AL-GR>

Abstract

Semantic identifiers (SIDs) have gained increasing attention in generative retrieval (GR) due to their meaningful semantic discriminability. However, current research on SIDs faces three main challenges: (1) the **absence of large-scale public datasets** with multimodal features, (2) limited investigation into optimization strategies for SID generation, which typically rely on costly GR training for evaluation, and (3) slow online convergence in industrial deployment. To address these challenges, we propose **FORGE**, a comprehensive benchmark for **FOR**ming semantic identifier in **Generative rEtrieval** with industrial datasets. Specifically, FORGE is equipped with a dataset comprising **14 billion user interactions** and multimodal features of **250 million items** sampled from Taobao, one of the biggest e-commerce platforms in China. Leveraging this dataset, FORGE explores several optimizations to enhance the SID construction and validates their effectiveness via offline experiments across different settings and tasks. Further online analysis conducted on the "Guess You Like" section of Taobao's homepage shows a 0.35% increase in transaction count, highlighting the practical impact of our method. Regarding the expensive SID validation accompanied by the full training of GRs, we propose **two novel metrics** of SID that correlate positively with recommendation performance, enabling convenient evaluations without any GR training. For real-world applications, FORGE introduces an offline pretraining schema that reduces online convergence by half. The code and data are available at https://github.com/selous123/al_sid.

1 Introduction

With the ability to predict the next item in an end-to-end manner, generative retrieval (GR) has recently emerged as a promising approach in recommender systems [1, 2]. Typically, this framework encodes user behaviors into a sequence of *identifiers*, employs larger models to capture item dependencies, and generates *identifiers* of candidates as results. Therefore, the tokenization, which determines identifiers for each item, plays a fundamental role in advancing accurate recommendation [3].

Recent advancements in item tokenization for recommendation can be broadly categorized into single-token identifiers and semantic identifiers (SIDs) [4, 5, 6]. **i)** In Figure 1(a), the former randomly assigns a unique identifier to each item (e.g., 1892392 $\rightarrow$ ) and updates the corresponding latent token independently during training. This schema would lead to the management of massive vocabu-

* Equal contribution. Work is done during Kairui Fu's internship at Alibaba Group.

† Corresponding authors.

larities with billions of items in real-world recommendations [7, 8]. Numerous identifiers prevent it from including all items, but only several negative ones are sampled for loss calculation to reduce computational burdens [9, 10]. Such an approximation, however, might result in inconsistency between

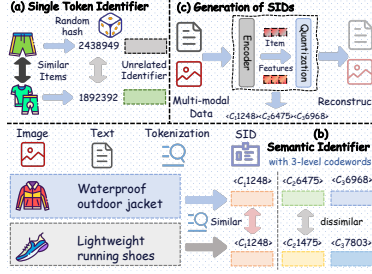


Figure 1: Diagram of (a) single token identifier and (b) semantic identifier. (c) The generation of SIDs.

offline training and online serving, where the latter requires measuring all items. **ii)** The semantic identifiers shown in Figure 1(b) represent items with multi-level **codewords** (e.g., $\langle C_1 1248 \rangle \langle C_2 6475 \rangle \langle C_3 6968 \rangle \rightarrow$ ) and retrieval models would generate the top-K item identifiers as candidates with continuous beam search [11, 6]. The construction of codewords within SID is based on multimodal data, enabling knowledge sharing among similar items. Furthermore, with hierarchical codewords, SIDs could compactly encode an exponentially large item space (e.g., a 3-level SID with 8192 tokens per level could represent $8192^3 \gg 10^{10}$ items using 24576 tokens). The small number of tokens also makes it computationally feasible to include all of them in the calculations during training [1].

As depicted in Figure 1(c), the tokenization process for SIDs typically follows an encoder-quantization framework: an encoder for extracting semantic features from raw multimodal data and another quantization module [12, 13] for SID encoding, separately [14]. These compact yet semantically rich identifiers are subsequently used by downstream retrieval models during task-specific training. Despite their promising results on real-world datasets [15, 5, 16], they still have the following challenges that hinder the development of this field:

1. **Limited behavior and features within datasets.** The majority of publicly available datasets in recommendation are limited in size (≤ 10 million) [17, 18], resulting in findings that may not generalize well when applied to industrial scenarios (≥ 1 billion) [19]. In addition, their construction is primarily based on single-token identifiers in the absence of abundant multimodal features, thus not accounting for SIDs [20, 21, 22]. The dataset limitations impede the ability of later researchers to generate practical insights.
2. **Neglect of the optimization for SID generation.** Even if SID serves as the foundation for the retrieval task [23, 16], useful insights around the design choices for it are typically not well-discussed in previous literature, making it complicated for researchers to select an optimal configuration. Furthermore, recent works assess the quality of SIDs through time-consuming training of GR [16], remaining a critical need for direct metrics with convenient SID evaluations.
3. **Inefficiency in industrial deployment.** Once a better SID configuration is confirmed via offline experiments, it typically requires deployment in the production environment and further training with the retrieval model for several days to reach convergence. The lengthy process hinders engineers from leveraging online observations to validate the SID optimizations adequately. Unfortunately, recent studies [24, 25] consistently overlook the practical aspects for efficient deployment. This absence limits their broader impact and applicability in real industrial settings.

To mitigate this gap, we propose **FORGE** , a comprehensive benchmark for **FOR**ming semantic identifie**R** in **Generative rE**trieval with industrial datasets. **(Challenge 1)** In contrast to current public datasets that suffer from small scale and insufficient multimodal richness, FORGE is collected from one of the biggest e-commerce platforms in China, providing more than 14 billion user behaviors in 10 days, along with multimodal features of 250 million items. We partition the dataset into three consecutive phases for continuous training and robustly separated evaluation, facilitating convenient usage for researchers. **(Challenge 2)** With the proposed dataset, FORGE introduces several optimization strategies for SID generation, encompassing diverse data modalities and ID collision mitigation (i.e., cases where multiple similar items correspond to the same SID). We thoroughly validate the effectiveness of these strategies across a range of configurations, including different downstream GR models, SID structures, and search tasks. Following improvements observed in the 7-day online evaluation further support the validity of the proposed optimizations. In addition, FORGE introduces two novel metrics (i.e., embedding hitrate and Gini coefficient) for directly assessing SID quality. Experiments show that these metrics exhibit strong correlation with the performance of GRs, enabling SID measurements without the need for costly GR training and thus providing an effective and efficient evaluation process. **(Challenge 3)** Serving as an industrial benchmark, FORGE also explores several warm-up techniques aimed at accelerating the convergence

of new SIDs deployed in online scenarios. Continuous experiments spanning 10 days indicate that employing an offline pre-training allows the model to reach the same level as the base production recommenders in only half the time. The major contributions of our paper are summarized as follows:

1. To the best of our knowledge, FORGE is the first generative retrieval benchmark with semantic identifiers built from industrial-scale user behaviors with rich multimodal features in Taobao. Specifically, FORGE contains 100 times more user interactions than the largest existing recommendation dataset, along with 26 times more users and 3 times more items.
2. To obtain a better SID for GR, we incorporate more valuable modalities with collaborative relations into the generation and propose several post-processing algorithms for ID collision. Furthermore, we propose two novel metrics that positively correlate with GR performance, facilitating cost-free SID quality assessment without any GR training.
3. We conduct extensive offline experiments to validate the effectiveness of our optimizations under different settings. Subsequent 0.35% online increase in transaction count within the "Guess You Like" section of Taobao further strengthens our credibility. As for the industrial serving, FORGE discovers an offline pretraining approach to accelerate the convergence of new SIDs in production.

2 Dataset

Table 1: Dataset statistics and properties. FORGE contains a large-scale, temporally continuous dataset with multimodal content and a hierarchical codebook for semantic modeling.

Dataset	Temporal Continuity	#Users	#Items	#Interactions	Modalities	Codebook	Tasks
MovieLens	N	138K	27K	20M	ID, text	N	Recsys
Yelp	N	2.1M	160K	8M	ID, text	N	Recsys
Taobao	N	987K	4M	100M	ID	N	Recsys
TenRec	N	5.0M	3.7M	142M	ID	N	Recsys
KuaiRec	N	7K	10K	12M	ID, tags	N	Recsys
RecFlow	N	42K	82M	38M	ID	N	Recsys
FORGE	Y	131M	251M	14B	ID, text, image	Y	Recsys, Search

2.1 Description

FORGE pioneers the first industrial dataset in recommendation containing a set of pre-processed SIDs directly adopted for generative retrieval, as well as the multimodal features used to generate these identifiers. Collected from one of the largest e-commerce platforms in China, it is designed to bridge the gap between research and industry by enabling insights derived from it to be validated in industrial settings. Specifically, the dataset in FORGE contains two main types, where the examples are placed in the Appendix A.4:

Seq Data samples 40 million user data per day over ten consecutive days for training. These interactions are partitioned into three sequential stages (S1, S2, and S3) based on their temporal order. For a better warm-up, the first stage S1 contains four days, while S2 and S3 own three days. For each training stage, a corresponding test set of 100,000 user sequences is sampled from the following day.

Each sequence is identified by the universally unique identifier of each page view (i.e., recommendation list presented to users). It mainly contains three fields: **i)** *action_seq* represents the sequence of historical interactions occurring before the current page view. To ensure efficient inference and compatibility with online deployment, we limit the maximum length of each sequence to 100 and truncate those longer sequences. **ii)** *target_item* refers to the items interacted with on the current page. For training, only the first clicked item is treated as the target. In contrast, all interacting items are considered as targets to compute metrics more comprehensively. **iii)** *query* is None for recommendation, while set to the corresponding query keywords for the search task, which are also collected from the real user inputs.

Item Info. Following previous benchmarks [22, 26], each item in *Seq Data* is represented by a single-token identifier (item ID). To standardize the generation of SIDs, FORGE enriches each item ID with additional multimodal information, serving as input for the encoding processes. To preserve privacy, we release only the processed *multimodal embeddings* and do not include any raw data.

Additionally, the item i^+ with the most collaborative relations of each item i ($i2i$ for simplicity) is provided as a field of each item and aids in SID generation in Section 3.1. For a better understanding of items, we employ named entity recognition to extract *keywords from item attributes*. We also release *constructed SIDs* in our open dataset, allowing better reproducibility by the community.

With these released data, researchers can flexibly construct their own SIDs, evaluate them using the proposed metrics in FORGE, or leverage the provided SIDs to train downstream recommendation or search models directly.

2.2 Analysis and Comparison

As depicted in Table 1, FORGE distinguishes itself from existing recommendation benchmarks in two aspects. On the one hand, early datasets such as MovieLens-20M [17] and Yelp [27] are relatively small in scale, with limited user-item interactions that make them unsuitable for modeling large-scale traffic typical of industrial systems. Subsequent datasets [26, 22] alleviate this with abundant interactions from real-world platforms. In Table 1, FORGE further significantly extends the largest dataset, TenRec, by approximately 100 times, providing over 14 billion user behaviors. The inclusion of multiple time periods for training and testing ensures temporal robustness and generalization. On the other hand, FORGE pioneers the SID construction in generative retrieval. In contrast to prior datasets that typically include only one or two modalities and cover limited items, FORGE integrates three complementary modalities across billions of items, enabling more comprehensive and diverse SID construction. Furthermore, the accompanying query attributes allow our benchmark to be naturally adapted for evaluating search tasks with minimal additional effort.

3 Overall Framework

This section provides an overview of FORGE, as presented in Figure 2. Specifically, it begins with SID generation (Section 3.1), followed by Section 3.2, which presents the post-processing procedure to resolve SID collisions. In Section 3.3, we introduce the generative retrieval, along with the evaluation metrics for both SID and recommendation in Section 3.4.

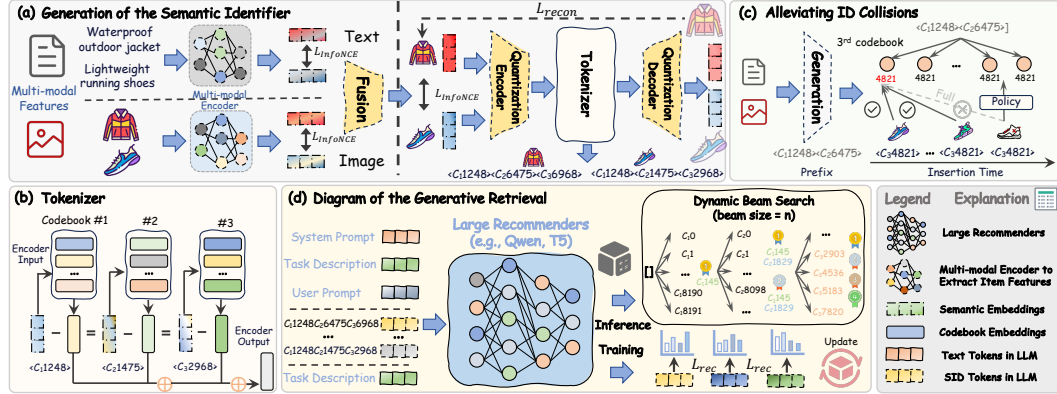


Figure 2: Overview of FORGE. (a) The generation process of SIDs. Left panel: the training of the text/image encoder. Right panel: the schematic diagram of item tokenization. (b) Detailed workflow of the tokenizer. (c) Post-collision handling for semantic identifiers. (d) The training and dynamic inference procedure of generative retrieval.

3.1 SID Generation

Effectively capturing the underlying semantics of items through their multimodal content is a key step in generating high-quality SIDs. A natural approach is to leverage pretrained multi-modal large language models [28], $\mathcal{M}_{\text{text}}$ and $\mathcal{M}_{\text{image}}$ in Figure 2(a), to extract and fuse features $\mathcal{H}^i$ from text $\mathcal{I}_{\text{text}}^i$ and image $\mathcal{I}_{\text{image}}^i$ of item i , respectively:

$$\mathcal{H}^i = \mathcal{M}_{\text{Fusion}}(\mathcal{H}_{\text{text}}^i, \mathcal{H}_{\text{image}}^i) = \text{Multimodal Fusion}(\mathcal{M}_{\text{text}}(\mathcal{I}_{\text{text}}^i), \mathcal{M}_{\text{image}}(\mathcal{I}_{\text{image}}^i)). \quad (1)$$

To further enhance the discriminability of these representations and reflect meaningful patterns from user behaviors, we introduce another contrastive loss L_{InfoNCE} . It encourages similar items that

frequently co-occur to have more similar text and image representations:

$$L_{\text{InfoNCE}} = f(\mathcal{H}_{\text{text}}^i, \mathcal{H}_{\text{text}}^{i+}, \mathcal{H}_{\text{text}}^{i-}) + f(\mathcal{H}_{\text{image}}^i, \mathcal{H}_{\text{image}}^{i+}, \mathcal{H}_{\text{image}}^{i-}) + f(\mathcal{H}^i, \mathcal{H}^{i+}, \mathcal{H}^{i-}), \quad (2)$$

where f denotes the InfoNCE loss [29], where the positive i^+ and negative i^- samples are collected from i2i in *Item Info* and in-batch negative sampling, respectively.

Item Tokenization. The multi-modal feature $\mathcal{H}^i$ of item i is then used to tokenize its SID. To ensure compatibility with the supervised learning framework and the autoregressive nature of generative retrieval, we employ the RQ-VAE [13] in Figure 2(b) to convert $\mathcal{H}^i$ into a sequence of discrete codewords $\{c_1, \dots, c_m\}$, with m as the number of codewords. Specifically, $\mathcal{H}^i$ is processed via the quantization encoder $\mathcal{E}$ into $z_1 \in R^d$. At each level $j \in [1, m]$, a corresponding codebook $\mathcal{B}_j = \{r_j^0, r_j^1, \dots, r_j^{n_j-1}\} \in R^{n_j \times d}$ is maintained, where n_j is the codebook size. At the first level ($j = 1$), z_1 is matched against all entries in $\mathcal{B}_1$ using nearest neighbor search. The selected codeword $c_1 = \arg \min_c \|z_1 - r_1^c\|$ corresponds to the representative vector $r_1^{c_1}$, which is subtracted from z_1 to compute the residual z_2 for the next level. Similarly, the second codeword c_2 is obtained by searching the nearest $r_2^{c_2}$ in $\mathcal{B}_2$ with z_2 . This procedure is repeated until all m codewords are determined.

To prevent the quantization from degenerating into mere ID assignment, we employ a reconstruction loss $\mathcal{L}_{\text{recon}}$ that minimizes the discrepancy between the quantization decoder $\mathcal{D}$ output given quantized representation $\sum_{j=1}^m r_j^{c_j}$ and the original feature $\mathcal{H}^i$:

$$\mathcal{L}_{\text{recon}} = \|\mathcal{D}(r_1^{c_1} + \dots + r_m^{c_m}) - \mathcal{H}^i\|_2. \quad (3)$$

In Appendix A.7, we compare the RQ-VAE with alternatives like random assignment, multiple VQ [12], and RQ-Kmeans [24] to validate the effectiveness.

3.2 Alleviating ID Collisions

During the SID generation, multiple similar items from different configurations or shops may be assigned the same SID, a situation referred to as **ID collision**. This may also result in low utilization, where certain SIDs remain unassociated with any item. To alleviate this issue, in Figure 2(c), we introduce two optional post-processing strategies to control the maximum number of items mapped to each SID. The **KNN-based** strategy samples multiple candidate SIDs and evaluates them sequentially based on their associated scores until a SID with fewer than σ corresponding items is found, where σ is a predefined threshold. In contrast, the **random-based** strategy focuses on dispersing the SID distribution at the final level rather than preserving semantic consistency. It assigns incremental codebooks to the last level in a circular manner (e.g., $\mathbf{C}_m \mathbf{0} \rightarrow C_m 1 \rightarrow \dots \rightarrow C_m n_m - 1$ (the last codeword) $\rightarrow \mathbf{C}_m \mathbf{0}$), following the insertion order of items sharing the same prefix. Both approaches lead to a fairer distribution of codebook usage and further improve the performance of GR.

3.3 Generative Retrieval

Once the set of SIDs is constructed, we concatenate the SIDs derived from user behavior with system instructions and user-specific prompts to form the input x for the large recommender, which can be implemented with arbitrary architectures like Qwen [30] or T5 [31]. Tokens of codewords are extended into the tokenizer τ and jointly trained with the recommender θ . Formally, given a sequence of codewords $\{c_1, c_2, \dots, c_m\}$ representing the SID of the target item, we use cross-entropy loss [32] to optimize both θ and τ by maximizing the conditional probability distribution $P_{\theta, \tau}(c_i | x, c_{j < i})$:

$$\mathcal{L}_{\text{rec}} = \sum_{i=1}^m \log P_{\theta, \tau}(c_i | x, c_{j < i}). \quad (4)$$

For inference, we employ beam search [33] and retain the top-K SIDs iteratively as the retrieval results. However, maintaining a fixed beam width across all decoding steps can introduce significant computational overhead, particularly in real-time scenarios involving billions of concurrent users. To address this, as illustrated in Figure 2(d), FORGE is equipped with **dynamic beam search**, which adaptively increases the beam size during generation (e.g., $300 \rightarrow 600 \rightarrow 1200$), thereby balancing computational efficiency and generation quality.

3.4 Evaluation Metrics

In this paper, we adopt the recommendation performance metric **HitRate** (HR) to evaluate the quality of generated SIDs. However, computing HR metrics requires full model training, which is typically resource-intensive and time-consuming. To address this, FORGE introduces **embedding hitrate** and **Gini coefficient** to directly assess the SID quality. Specifically, embedding hitrate provides a preliminary evaluation of multimodal embeddings $\mathcal{H}^i$, while the Gini coefficient measures the fairness or utilization of codebooks. Our experiments demonstrate that both metrics exhibit a strong correlation with HR, enabling an efficient SID evaluation through these two metrics without the need for additional GR training. Further analysis and detailed descriptions are provided in Appendix A.6.

4 Experiments

In this section, we conduct experiments on our proposed datasets across three consecutive stages (S1, S2, and S3). For validation, we sample data from the day following each stage. By default, we employ RQ-VAE for SID generation and **Qwen2.5-0.5B** for generative retrieval. The goal of these experiments is to address the following research questions:

- **RQ1:** What generation strategies of SIDs are beneficial for downstream generative retrieval in terms of the hitrate?
- **RQ2:** In addition to the performance of GR accessed after time-consuming iteration, what evaluation criteria can be used to judge the quality of the SIDs more conveniently?
- **RQ3:** How is the generalization of SID optimization strategies proposed in this paper?
- **RQ4:** In practice, once a better SID configuration is identified offline in applications, how can it be deployed online to achieve fast convergence and obtain an effective observation?

Table 2: The effectiveness of SIDs optimization proposed in FORGE compared with base version (3×8192 as the 3-level codebooks) where **bold** represents the best method.

Stage	Method	HR@20		HR@100		HR@500		HR@1000	
S1	base	3.61%	+0.00%	9.67%	+0.00%	20.82%	+0.00%	25.09%	+0.00%
	+KNN-10	3.91%	+8.31%	10.32%	+6.72%	21.85%	+4.95%	25.38%	+1.16%
	+KNN-5	4.18%	+15.79%	10.81%	+11.79%	22.14%	+6.34%	25.30%	+0.84%
	+Random-5	4.67%	+29.36%	11.79%	+21.92%	23.65%	+13.59%	26.30%	+4.82%
	+i2i	3.79%	+4.99%	10.04%	+3.83%	21.71%	+4.27%	26.97%	+7.49%
	+sideinfo	3.81%	+5.54%	10.00%	+3.41%	21.69%	+4.18%	26.88%	+7.13%
	Ours	4.89%	+35.46%	12.31%	+27.30%	24.79%	+19.07%	29.01%	+15.62%
S2	base	4.15%	+0.00%	10.70%	+0.00%	22.71%	+0.00%	26.70%	+0.00%
	+KNN-10	4.63%	+11.57%	11.52%	+7.66%	23.24%	+2.33%	26.80%	+0.37%
	+KNN-5	4.81%	+15.90%	11.90%	+11.21%	24.18%	+6.47%	27.09%	+1.46%
	+Random-5	5.23%	+26.02%	12.88%	+20.37%	25.05%	+10.30%	27.74%	+3.90%
	+i2i	4.12%	-0.72%	10.69%	-0.09%	23.06%	+1.54%	28.10%	+5.24%
	+sideinfo	4.32%	+4.10%	11.06%	+3.36%	23.46%	+3.30%	28.43%	+6.48%
	Ours	5.33%	+28.43%	13.23%	+23.64%	26.37%	+16.12%	30.28%	+13.41%
S3	base	4.33%	+0.00%	11.16%	+0.00%	23.26%	+0.00%	27.24%	+0.00%
	+KNN-10	4.56%	+5.31%	11.78%	+5.56%	24.06%	+3.44%	27.54%	+1.10%
	+KNN-5	5.08%	+17.32%	12.61%	+12.99%	24.66%	+6.02%	27.72%	+1.76%
	+Random-5	5.43%	+25.40%	13.14%	+17.74%	25.39%	+9.16%	28.03%	+2.90%
	+i2i	4.22%	-2.54%	11.32%	+1.43%	23.69%	+1.85%	28.91%	+6.13%
	+sideinfo	4.34%	+0.23%	11.43%	+2.42%	24.20%	+4.04%	29.32%	+7.64%
	Ours	5.44%	+25.64%	13.79%	+23.57%	26.71%	+14.83%	30.86%	+13.29%

4.1 Overall Analysis about the Optimization of SIDs (RQ1)

Table 2 presents the ablation results based on the 3-level codewords with 8192 codebooks per level (i.e., 3×8192). Base is constructed using limited item-to-item collaborative information as defined in

Equation 2, along with a KNN-based post-processing where the threshold σ is set to 25. From the results, we can conclude the following findings from two perspectives: **i) Post-processing to reduce collisions after SID generation leads to a more fair SID distribution and significantly improves retrieval performance.** +KNN-5 and +KNN-10 impose a limit on the number of items each SID can represent using the KNN-based solution. Relative better metrics from base to +KNN-10 and +KNN-5 confirm the benefit of increasing fairness and representation granularity. In contrast, +Random-5 randomly assigns only the last codeword of each SID to an item, irrespective of semantic similarity. This method yields a more balanced SID distribution and further boosts retrieval performance, suggesting that the final layer of SIDs is less critical than maintaining high utilization or fairness across the SID space. **ii) Incorporating higher-quality modalities enhances the precision of SID construction.** The +i2i variant introduces co-occurrence-based item relationships into the contrastive learning framework of Equation 2, while +sideinfo enriches the SID generation process with additional textual item metadata (e.g., categories, sellers). Although +i2i slightly underperforms the base model in terms of HR@20, both strategies outperform the baseline at larger recall thresholds (e.g., HR@1000), achieving over 5% improvements consistently across all training stages. This highlights the value of richer input signals in enhancing the effectiveness of GR. By integrating all these strategies, ours using more sideinfo and i2i data with Random-5 for SID generation demonstrates consistently more than 13% increase in hitrate compared to base.

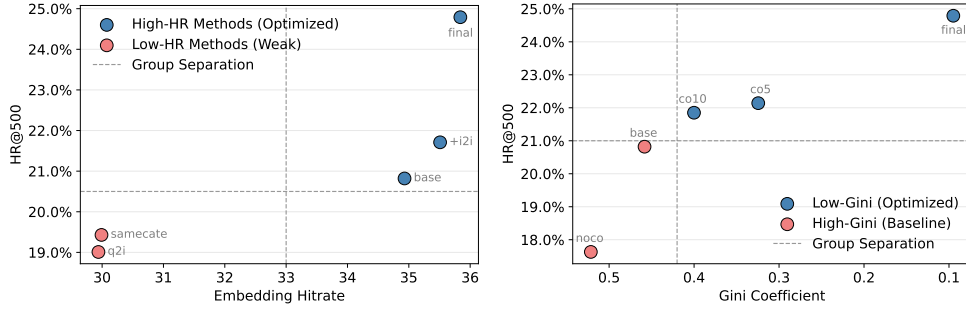


Figure 3: The effectiveness of embedding hitrate and Gini coefficient for SID evaluation. Left: Positive correlation between embedding hitrate and retrieval performance. Right: Lower Gini coefficient (fairer SID usage) leads to higher HR@500.

4.2 Direct Metrics for the Evaluation of SIDs (RQ2)

The direct evaluations of SIDs using embedding hitrate and Gini coefficient are presented in Figure 3. As we mentioned in Section 3.4, the embedding hitrate captures the quality of item collaborative relationships within the multi-modal feature $\mathcal{H}^i$, while the Gini coefficient reflects the fairness of SID distribution. **First**, it is evident that leveraging more related information could lead to the improvement of the associated metrics. From the figure 3, we could observe that incorporating richer relational information consistently improves these metrics. For instance, *i2i* with more collaborative item-item interactions achieves a higher embedding hitrate compared to the baseline base and methods like query2i. Likewise, a more refined SID collision strategy (e.g., *noco*→*base*→*KNN-10*→*KNN-5*→*Random-5*) could lead to higher codebook fairness and thus a lower Gini coefficient. We **then** train the retrieval model with all these SIDs across three stages, and surprisingly discover that both metrics have a strong relation with the ultimate hitrate. This motivates us that **it is not strictly necessary to train recommenders** to assess the quality of SIDs. Instead, once a new set of SIDs is generated through the encoder-quantization framework, our proposed metrics could directly provide a reliable and efficient proxy for evaluating SID effectiveness without GR training, offering a practical and scalable solution for future research and applications.

4.3 Analysis about the Generalization of SID Optimization (RQ3)

All previous experiments were conducted using 3-level SIDs with 8192 codebooks at each level and the decoder-only architecture Qwen2.5-0.5B. In this part, we are interested in whether the proposed optimizations are robust across different configurations, architectures, and domains. Therefore, we conduct additional experiments that explore variations in these dimensions.

Impact of the Level of Codebook. The comparison between base and Ours using 2-level codewords (i.e., 32768 codebooks each level) is placed in Table 3. This setup maintains the total representational capacity of the original multi-level design but distributes it across fewer levels. The results verify that our optimization also generalizes well to configurations with fewer hierarchical levels. Across three stages, ours achieves significant improvements over base, with more than a 30% increase in HR@10 and a 14% increase in HR@1000.

Table 3: Performance of base and Ours under 2×32768 level structure.

Stage	Version	HR@20	HR@100	HR@500	HR@1000
S1	base	3.37%	9.09%	20.33%	26.94%
	Ours	4.57%	11.94%	24.67%	31.34%
S2	base	3.85%	10.03%	21.93%	28.96%
	Ours	5.08%	12.61%	26.17%	33.53%
S3	base	3.91%	10.49%	22.65%	30.00%
	Ours	5.11%	13.04%	26.88%	34.15%

Table 4: Performance of Ours under different architecture and model size in Stage S1.

Codebook	t5-base (0.2B)				Qwen2.5-3B			
	HR@20	HR@100	HR@500	HR@1000	HR@20	HR@100	HR@500	HR@1000
3×8192 (base)	1.29%	3.62%	8.91%	11.98%	4.49%	11.71%	24.40%	28.94%
3×8192 (Ours)	2.36%	5.99%	12.98%	16.08%	5.60%	14.18%	27.58%	32.12%
2×32768 (base)	1.84%	5.14%	11.98%	16.45%	3.72%	10.19%	22.64%	29.75%
2×32768 (Ours)	2.46%	6.55%	14.30%	18.88%	4.96%	12.80%	27.13%	34.29%

Analysis of the Architecture. The left panel of Table 4 presents the performance of t5-base [34] with an encoder-decoder architecture. While this architecture differs from the decoder-only setup we used in previous experiments, our proposed optimizations still yield substantial improvements across different architectural paradigms. The consistent performance gain over the base model for both types of SIDs highlights our broad applicability.

Influence of the Model Size. The ability of FORGE can also be extended to larger models with 3B parameters, shown in the right part of Table 4. Moreover, increasing the model size consistently enhances retrieval performance, leading to nearly 10% improvements across all settings compared to the Qwen2.5-0.5B baseline in Table 2 and 3. We attribute this to the greater knowledge retention capacity enabled by the expanded parameter count.

Table 5: Retrieval performance on the generative search task across stages. Ours consistently outperforms the base counterparts under both codebook configurations.

Stage	Codebook	HR@20	HR@100	HR@500	HR@1000
S1	3×8192 (base)	14.30%	27.91%	43.95%	50.74%
	3×8192 (Ours)	23.39%	37.05%	51.77%	56.89%
	2×32768 (base)	11.53%	24.09%	39.65%	47.21%
	2×32768 (Ours)	17.77%	31.56%	46.98%	53.91%
S2	3×8192 (base)	14.93%	28.98%	46.21%	53.10%
	3×8192 (Ours)	24.20%	38.76%	54.08%	59.36%
	2×32768 (base)	12.14%	25.38%	41.77%	49.47%
	2×32768 (Ours)	18.89%	32.98%	49.14%	56.10%
S3	3×8192 (base)	15.08%	29.83%	47.83%	55.10%
	3×8192 (Ours)	24.71%	40.06%	56.04%	61.46%
	2×32768 (base)	12.54%	26.26%	43.48%	51.57%
	2×32768 (Ours)	19.13%	34.19%	51.18%	58.34%

Generalization to the Search Domain. In industrial scenarios, mobile apps often involve multiple tasks such as search and recommendation. Building a separate SID for each task can be resource-intensive. In this section, we investigate whether SIDs trained on the recommendation task can generalize to the search task. The input for the search task presented in Table 10 is similar to the recommendation, except for the query keywords of each user. As shown in Table 5, our findings in the search domain align closely with those observed in recommendation, where the 3-layer SID consistently outperforms the 2-layer SID. Moreover, across all configurations, the proposed SID

optimization strategy demonstrates clear superiority over the baseline. This phenomenon highlights the potential of FORGE for enabling versatile SIDs in real-world deployment scenarios.

Online Study. The improvement from a 7-day online experiment conducted on the Guess You Like section of Taobao’s homepage is presented in Table 6, serving as strong evidence of our optimizations, where our optimization improves the PVR and HR by 8.93% and 10.02% compared to base. Moreover, replacing base with ours ultimately results in a 0.35% absolute increase in user transactions.

Table 6: Online improvement with our SID optimization from 1st Sep to 7th Sep.

PVR	Hirate	Transaction Count
+8.93%	+10.02%	+0.35%

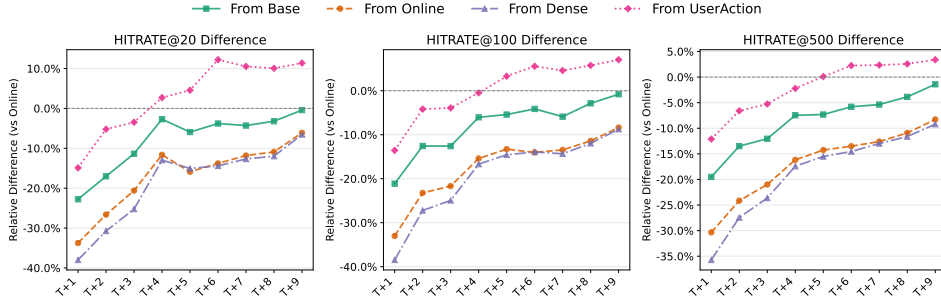


Figure 4: Relative difference from the production base version across cold-start days.

4.4 Practical Insights for Codebook Upgrade and Convergence (RQ4)

In contrast to offline training, models deployed in real-world production systems typically require training on daily-emerging behaviors (≥ 200 million samples each day) for tens of days until convergence. Once we find a more powerful SID, thoroughly verifying it through comparison with production models requires much effort. To address this challenge, we propose several initialization strategies aimed at accelerating the convergence.

Specifically, *From Base* initializes the training using Qwen2.5, while *From Online* starts from an existing production model equipped with SIDs of the same size (e.g., using the tokenizer and weights of the base as initialization for Ours). Following Qwen-VL [35], *From Dense* assumes that knowledge encoded in the production model is beneficial for the new SID. It retains the weights from the production model but independently initializes the SID-related tokens. Finally, *From UserAction* performs offline pretraining using concatenated user behaviors from the previous 7 days, followed by streaming training. To ensure a fair comparison, all methods except *From UserAction* are additionally deployed online starting from days T-1 and T-2. Further details can be found in Appendix A.4.

Figure 4 illustrates the evolution of online performance across several days. The *From Base* strategy catches up with the production model after 10 days of training and requires even more time to outperform the baseline. Initializing from the production model does not consistently benefit the new SID, both *From Online* and *From Dense* underperform relative to *From Base*. With thorough pretraining, *From UserAction* surpasses the production model within four days and achieves a 10% improvement in Hitrate@10 after ten days of training. This demonstrates its potential to significantly accelerate deployment in practical settings.

5 Conclusion

In this paper, we introduce **FORGE**, a comprehensive benchmark for **FO**rming semantic identifier in **GE**nerative **rE**trieval with industrial datasets. FORGE is the first industrial benchmark containing 14 billion user interactions and multimodal features of 250 million items for semantic identifier construction, offering a robust foundation for advancing research in semantic identifier construction. Leveraging this, we systematically explore significant yet less explored configurations of SIDs and propose several optimizations to improve their quality. Extensive offline and online results validate the effectiveness of our approach. Additionally, we present a practical pretraining schema that enables faster convergence in industrial settings. We hope that FORGE will serve as a valuable resource for the community, encouraging further innovation in this area.

References

- [1] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems*, 36:10299–10315, 2023.
- [2] Zida Liang, Changfa Wu, Dunxian Huang, Weiqiang Sun, Ziyang Wang, Yulian Yan, Jian Wu, Yuning Jiang, Bo Zheng, Ke Chen, et al. Tbgrecall: A generative retrieval model for e-commerce recommendation scenarios. *arXiv preprint arXiv:2508.11977*, 2025.
- [3] Haohao Qu, Wenqi Fan, Zihuai Zhao, and Qing Li. Tokenrec: Learning to tokenize id for llm-based generative recommendations. *IEEE Transactions on Knowledge and Data Engineering*, 2025.
- [4] Xinyu Lin, Haihan Shi, Wenjie Wang, Fuli Feng, Qifan Wang, See-Kiong Ng, and Tat-Seng Chua. Order-agnostic identifier for large language model-based generative recommendation. In *Proceedings of the 48th international ACM SIGIR conference on research and development in information retrieval*, pages 1923–1933, 2025.
- [5] Enze Liu, Bowen Zheng, Cheng Ling, Lantao Hu, Han Li, and Wayne Xin Zhao. Generative recommender with end-to-end learnable item tokenization. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 729–739, 2025.
- [6] Bowen Zheng, Hongyu Lu, Yu Chen, Wayne Xin Zhao, and Ji-Rong Wen. Universal item tokenization for transferable generative recommendation. *arXiv preprint arXiv:2504.04405*, 2025.
- [7] Oren Barkan and Noam Koenigstein. Item2vec: neural item embedding for collaborative filtering. In *2016 IEEE 26th international workshop on machine learning for signal processing (MLSP)*, pages 1–6. IEEE, 2016.
- [8] Aleksandr V Petrov, Craig Macdonald, and Nicola Tonellotto. Efficient recommendation with millions of items by dynamic pruning of sub-item embeddings. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2102–2111, 2025.
- [9] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152*, 2024.
- [10] Haokai Ma, Ruobing Xie, Lei Meng, Fuli Feng, Xiaoyu Du, Xingwu Sun, Zhanhui Kang, and Xiangxu Meng. Negative sampling in recommendation: A survey and future directions. *arXiv preprint arXiv:2409.07237*, 2024.
- [11] Kaiyuan Li, Rui Xiang, Yong Bai, Yongxiang Tang, Yanhua Cheng, Xialong Liu, Peng Jiang, and Kun Gai. Bbqrec: Behavior-bind quantization for multi-modal sequential recommendation. *arXiv preprint arXiv:2504.06636*, 2025.
- [12] Patrick Esser, Robin Rombach, and Bjorn Ommer. Taming transformers for high-resolution image synthesis. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12873–12883, 2021.
- [13] Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. Autoregressive image generation using residual quantization. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11523–11532, 2022.
- [14] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. Adapting large language models by integrating collaborative semantics for recommendation. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 1435–1448. IEEE, 2024.
- [15] Yidan Wang, Zhaochun Ren, Weiwei Sun, Jiyuan Yang, Zhixiang Liang, Xin Chen, Ruobing Xie, Su Yan, Xu Zhang, Pengjie Ren, et al. Enhanced generative recommendation via content and collaboration integration. *CoRR*, 2024.
- [16] Clark Mingxuan Ju, Liam Collins, Leonardo Neves, Bhuvesh Kumar, Louis Yufeng Wang, Tong Zhao, and Neil Shah. Generative recommendation with semantic ids: A practitioner’s handbook. *arXiv preprint arXiv:2507.22224*, 2025.

- [17] F Maxwell Harper and Joseph A Konstan. The movielens datasets: History and context. *Acm transactions on interactive intelligent systems (tiis)*, 5(4):1–19, 2015.
- [18] Jianmo Ni, Jiacheng Li, and Julian McAuley. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 188–197, 2019.
- [19] Chao Li, Zhiyuan Liu, Mengmeng Wu, Yuchi Xu, Huan Zhao, Pipei Huang, Guoliang Kang, Qiwei Chen, Wei Li, and Dik Lun Lee. Multi-interest network with dynamic routing for recommendation at tmall. In *Proceedings of the 28th ACM international conference on information and knowledge management*, pages 2615–2623, 2019.
- [20] Jieming Zhu, Jinyang Liu, Shuai Yang, Qi Zhang, and Xiuqiang He. Open benchmarking for click-through rate prediction. In *Proceedings of the 30th ACM international conference on information & knowledge management*, pages 2759–2769, 2021.
- [21] Jieming Zhu, Quanyu Dai, Liangcai Su, Rong Ma, Jinyang Liu, Guohao Cai, Xi Xiao, and Rui Zhang. Bars: Towards open benchmarking for recommender systems. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2912–2923, 2022.
- [22] Guanghu Yuan, Fajie Yuan, Yudong Li, Beibei Kong, Shujie Li, Lei Chen, Min Yang, Chenyun Yu, Bo Hu, Zang Li, et al. Tenrec: A large-scale multipurpose benchmark dataset for recommender systems. *Advances in Neural Information Processing Systems*, 35:11480–11493, 2022.
- [23] Liu Yang, Fabian Paischer, Kaveh Hassani, Jiacheng Li, Shuai Shao, Zhang Gabriel Li, Yun He, Xue Feng, Nima Noorshams, Sem Park, et al. Unifying generative and dense retrieval for sequential recommendation. *arXiv preprint arXiv:2411.18814*, 2024.
- [24] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. Onerec: Unifying retrieve and rank with generative recommender and iterative preference alignment. *arXiv preprint arXiv:2502.18965*, 2025.
- [25] Zuowu Zheng, Ze Wang, Fan Yang, Jiangke Fan, Teng Zhang, Yongkang Wang, and Xingxing Wang. Ega-v2: An end-to-end generative framework for industrial advertising. *arXiv preprint arXiv:2505.17549*, 2025.
- [26] Qi Liu, Kai Zheng, Rui Huang, Wuchao Li, Kuo Cai, Yuan Chai, Yanan Niu, Yiqun Hui, Bing Han, Na Mou, et al. Recflow: An industrial full flow recommendation dataset. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [27] Nabiha Asghar. Yelp dataset challenge: Review rating prediction. *arXiv preprint arXiv:1605.05362*, 2016.
- [28] An Yang, Junshu Pan, Junyang Lin, Rui Men, Yichang Zhang, Jingren Zhou, and Chang Zhou. Chinese clip: Contrastive vision-language pretraining in chinese. *arXiv preprint arXiv:2211.01335*, 2022.
- [29] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021.
- [30] Qwen Team. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.
- [31] Jianmo Ni, Gustavo Hernandez Abrego, Noah Constant, Ji Ma, Keith Hall, Daniel Cer, and Yinfei Yang. Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 1864–1874, 2022.
- [32] Shie Mannor, Dori Peleg, and Reuven Rubinfeld. The cross entropy method for classification. In *Proceedings of the 22nd international conference on Machine learning*, pages 561–568, 2005.
- [33] Markus Freitag and Yaser Al-Onaizan. Beam search strategies for neural machine translation. *arXiv preprint arXiv:1702.01806*, 2017.
- [34] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.

- [35] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond. *arXiv preprint arXiv:2308.12966*, 2023.
- [36] Jesús Bobadilla, Fernando Ortega, Antonio Hernando, and Abraham Gutiérrez. Recommender systems survey. *Knowledge-based systems*, 46:109–132, 2013.
- [37] Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. Deep learning based recommender system: A survey and new perspectives. *ACM computing surveys (CSUR)*, 52(1):1–38, 2019.
- [38] Qihang Yu, Kairui Fu, Shengyu Zhang, Zheqi Lv, Fan Wu, and Fei Wu. Thinkrec: Thinking-based recommendation via llm. *arXiv preprint arXiv:2505.15091*, 2025.
- [39] Wang-Cheng Kang and Julian McAuley. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*, pages 197–206. IEEE, 2018.
- [40] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. Session-based recommendations with recurrent neural networks. *arXiv preprint arXiv:1511.06939*, 2015.
- [41] Yidan Wang, Zhaochun Ren, Weiwei Sun, Jiyuan Yang, Zhixiang Liang, Xin Chen, Ruobing Xie, Su Yan, Xu Zhang, Pengjie Ren, et al. Content-based collaborative generation for recommender systems. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pages 2420–2430, 2024.
- [42] Wenjie Wang, Honghui Bao, Xinyu Lin, Jizhi Zhang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. Learnable item tokenization for generative recommendation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pages 2400–2409, 2024.
- [43] Yupeng Hou, Jiacheng Li, Ashley Shin, Jinsung Jeon, Abhishek Santhanam, Wei Shao, Kaveh Hassani, Ning Yao, and Julian McAuley. Generating long semantic ids in parallel for recommendation. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 956–966, 2025.
- [44] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. Optimized product quantization. *IEEE transactions on pattern analysis and machine intelligence*, 36(4):744–755, 2013.
- [45] Chongming Gao, Shijun Li, Wenqiang Lei, Jiawei Chen, Biao Li, Peng Jiang, Xiangnan He, Jiaxin Mao, and Tat-Seng Chua. KuaiRec: A fully-observed dataset and insights for evaluating recommender systems. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, pages 540–550, 2022.
- [46] Guorui Zhou, Hengrui Hu, Hongtao Cheng, Huanjie Wang, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Lu Ren, Liao Yu, et al. Onerec-v2 technical report. *arXiv preprint arXiv:2508.20900*, 2025.
- [47] Ben Chen, Xian Guo, Siyuan Wang, Zihan Liang, Yue Lv, Yufei Ma, Xinlong Xiao, Bowen Xue, Xuxin Zhang, Ying Yang, et al. Onesearch: A preliminary exploration of the unified end-to-end generative framework for e-commerce search. *arXiv preprint arXiv:2509.03236*, 2025.

A Appendix

A.1 The Use of Large Language Models (LLMs)

In this paper, we use publicly available LLMs as the backbone model of our experiments. As for writing, we use generative AI tools such as Grammarly solely to improve the clarity, coherence, and overall quality of the written text. All figures, preliminary drafts, and content are written independently by the authors without reliance on LLMs.

A.2 Related Work

Generative Retrieval. The rapid development of recommender systems has drawn a surge of research attention in recent years [36, 37, 38]. As the initial filtering component in the recommendation system, the retrieval stage quickly narrows down the item pool from billions to tens of thousands of candidates that users may be interested in. Traditional retrieval methods [39, 40, 19] encode users and candidate items into dense embeddings with a unified space, followed by Approximate Nearest Neighbors (ANN) to select several related items given each user. On the contrary, generative retrieval [1, 24, 25] directly decodes the identifiers of user-interested items in an auto-regressive way, introducing an innovative end-to-end approach for recommendation. Based on it, subsequent work adapted more suitable training frameworks [2, 5] and introduced strategies like reinforcement learning [24, 25] to generate items that better align with user preferences.

Semantic Identifier. The innovative SID proposed in TIGER [1] utilizes multiple codewords to build identifiers for items, enabling knowledge sharing among similar ones. From the perspective of modality to build SIDs, subsequent researchers claim that the collaborative information of items would contribute to better modeling of SIDs. Most of them [41, 5, 42, 11] involve extra knowledge from user-item interactions to align the multimodal features of items, making the generated SIDs equipped with both semantic and collaborative information in recommendation. As for the encoding, OneRec [24] and RPG [43] tend to simplify the quantization process and accelerate the decoding process via RQ-Kmeans and optimized product quantization (OPQ) [44]. In addition, FORGE focuses on the fairness and utilization of SID and codebooks. It provides more optimizations across all those perspectives, providing a comprehensive analysis of their influence with reliable empirical results. To the best of our knowledge, FORGE is the first industrial benchmark to analyze the significance of all these aspects in this community.

Dataset Benchmarks. In recent years, several public datasets [17, 27] have been introduced to support reproducible research by enabling fair comparisons under consistent settings. More recently, benchmarks such as Bars [21] have aimed to unify and standardize these datasets, streamlining evaluation across retrieval and ranking tasks in recommendation systems. Despite their popularity and contributions, most existing datasets suffer from limited user behaviors. For example, widely used collections like Amazon [18] typically contain fewer than 10 million interactions, with average user interaction sequences shorter than 20. These limitations hinder their applicability to large-scale industrial platforms that process hundreds of millions of daily actions. To address this gap, FORGE, alongside other industrial benchmarks [22, 26, 45], introduces massive datasets derived from real-world systems. Compared to prior efforts, FORGE distinguishes itself through its larger scale, stronger focus on generative retrieval with SID, and the integration of rich multimodal features.

A.3 Algorithm of the post-processing for ID collisions.

The pseudocodes of the KNN-based policy and the random-based policy are shown in Algorithm 1 and Algorithm 2, respectively.

A.4 Detailed Dataset Description

Seq Data. An example of our sequence data is presented in Table 7. Each interaction in the sequence is uniquely identified by the row number, representing a single page view from a user. The action_seq denotes the historical interactions used to generate the recommendation list for this page view. Items within each sequence are represented by item_ids and separated by commas. To enhance reliability, FORGE supports another dataset for the search task, whose data follows the structure of

Algorithm 1 KNN-based Policy for alleviating ID Collisions

```

1: codeword1, codeword2, {multiple_codewords3}  $\leftarrow$  QUANTIZATION( $\mathcal{H}^i$ )
2: for all codeword3  $\in$  {multiple_codewords3} do
3:   SID  $\leftarrow$  [codeword1][codeword2][codeword3]
4:   if count_items(SID)  $< \sigma$  then                                      $\triangleright \sigma$  is a predefined threshold
5:     return SID
6:   end if
7: end for

```

Algorithm 2 Random-based Policy for alleviating ID Collisions

```

1: Global IndexMap  $\leftarrow$  {}                                      $\triangleright$  Dictionary: key=(c1,c2), value=index
2: codeword1, codeword2  $\leftarrow$  QUANTIZATION( $\mathcal{H}^i$ )
3: key  $\leftarrow$  (codeword1, codeword2)
4: if key  $\notin$  IndexMap then
5:   IndexMap[key]  $\leftarrow$  0                                        $\triangleright$  Initialize index to 0 on first occurrence
6: end if
7: codeword3  $\leftarrow$  IndexMap[key]
8: IndexMap[key]  $\leftarrow$  (IndexMap[key] + 1) mod  $n_3$               $\triangleright n_3$  is the size of third-level codebook
9: return [codeword1][codeword2][codeword3]

```

the recommendation, except for the inclusion of an extra query keyword. All behaviors are sampled from a real-world platform, with 40 million items sampled per day over ten days. The data is divided into three stages (S1, S2, S3), where S1 contains four days for a better warm-up, while S2 and S3 contain three days. An additional 100,000 samples are collected on the following day after each stage for evaluation. The observed performance improvements across these three stages could provide strong empirical evidence of the effectiveness of research.

Table 7: Example of recommendation and search samples with segmented fields. For validation, all the interacted items during this page view are used as targets.

Task	Data split	pv_id	target_item	query	opt_seq
Recsys	train	d487e12a-689c-4332-825d-27efbceb9e72	835905354006	/	566476193770, 907995423965, 897259251809, 916138422508, 841619863136, 841665058110, 895390654577, 895382438483, 844457291108
	test	d492fea7-16fb-4652-943c-0b2abb383c1a	817445445744, 780409885810	/	546901080156, 626466181497, 804805887937, 819489082025, 919698707572, 680837256237, 611110770249, 696394590725, 738397879056
Search	train	0004350c2012e9af6d2b93e65aa593e3	762971121687	富勒烯精华液	937018558175, 936950943952, 947234504809, 666112772468, 928130569440, 938702048112, 940280943038, 918448888947, 792039516661
	test	001346c8d75a5144ddfa9589e3e27138	562133894238, 567812038293, 610069547271	干艾叶	736070049413, 949041120166, 945601927485, 922276261406, 801633068800, 901162945401, 884983761715, 705197027339, 890974760840

Item Info. The mapping between item_ids in the action_seq and their corresponding semantic identifiers (SIDs) is detailed in the Item Info of Table 8, where each entry provides relevant metadata associated with a given item_id. In addition to the constructed SIDs of each item i (base and Ours), the dataset includes its related item i^+ with the most co-occurrence relationship and the multimodal feature $\mathcal{H}^i$ derived by both the image and text of item i . To protect user privacy, FORGE does not make the actual title of items available. Instead, we employ named entity recognition (NER) to

extract meaningful keywords that serve as anonymized substitutes, facilitating the understanding of collaborative items while safeguarding sensitive information.

Table 8: Example of Item Info.

item_id	$\mathcal{H}^i \in R^{512}$	semantic_id	related_item	title
835905354006	[0.12, 0.56, ..., 0.03]	[1203,2315,3576]	787551011877	女装衬衫
855036080309	[0.04, 0.02, ..., 0.05]	[2706,4659,2176]	545092516562	大码睡衣男款

For Generative Retrieval Task. The instructions for the generative retrieval and search tasks are presented in Table 9 and Table 10, respectively. Each user behavior sequence is encoded as continuous SIDs with no separator. The inputs consist of both system and user instructions, while the answer is only made up of three codewords of the target SID. Optionally, for the search task, the query keywords of each request are added to the user instruction and placed after the user sequence. The whole training process is to enable LLMs to understand the instructions and provide satisfying item lists given an input.

Table 9: Example of the instruction-tuning dataset format for next-item prediction in e-commerce recommendation. Each behavior sequence is represented as a series of semantic IDs (e.g., C1220C8322C20452). For a better LLM tokenization, for the j th-level codeword in our processed SID, we represent it with $c_j + \sum_{k=1}^{j-1} n_k$ (e.g., C8193 with C_21 as the 2nd codeword and 8192 codebooks each level). The original data is in Chinese, and an English translation is provided below for reader comprehension.

Field	Content
system	你是一个推荐系统，根据用户的历史行为，预测用户在电商场景的下一步行为。我会给你一串连续行为的语义编码，按照用户点击的时间顺序排列，每个行为用一个语义ID表示。
user	当前用户的历史行为如下：C1220C8322C20452C6084C10195C20067C3256C14673C21112C7054C9412C18926C3021C10986C18869C3411C15297C23680C116C15928C19183C4405C11869C21320C7221C13888C16791C7743C16005C22091 请预测用户在电商推荐场景后续行为的语义编码。
answer	C3626C8758C22717

Translations are provided solely for reader comprehension:

- **system:** You are a recommendation system. Predict the user’s next action in an e-commerce scenario based on their historical behavior. A sequence of semantic IDs will be provided, representing past interactions in chronological order, with each behavior encoded as one semantic ID.
- **user:** User’s historical behavior sequence: [long sequence]. Please predict the semantic ID of the user’s next behavior in the e-commerce recommendation scenario.
- **answer:** C3626C8758C22717

For From UserAction Task. As described in Section 4.4, the best-performed initialization is achieved through offline continue pretraining using interactions collected over the past 7 days. As the objective of the pretraining is to enable LLMs to swiftly adjust to evolved SIDs, we omit the instruction parts but only retain tokens in the answer. Specifically, all user behaviors during this period are organized sequentially based on the interaction time and optimized in an autoregressive way in Table 11. This offline pretraining allows LLMs to quickly grasp the structure and collaborative information of new SIDs, significantly accelerating convergence during online deployment.

A.5 Implementation Details

Running environment. The experiments are conducted on the PPU 810E, a GPU architecture developed by Alibaba. This platform achieves approximately 90% of the computational performance

Table 10: Example of the instruction-tuning dataset format for next-item prediction in e-commerce search. Each behavior sequence is represented as a series of semantic IDs (e.g., C1220C8322C20452). The original data is in Chinese, and an English translation is provided below for reader comprehension.

Field	Content
system	你是一个搜索系统，根据用户的历史行为和当前用户搜索词，预测用户在电商场景的下一步行为。我会给你用户的搜索词和一串连续行为的语义编码，按照用户点击的时间顺序排列，每个行为用三个词表示。
user	当前用户的历史行为如下：C5299C12905C22291C1477C12826C20935C1931C9821C24180C853C14294C22645C2075C12389C16385C853C13340C20935C4894C14014C18136C6398C13220C22786C6398C12675C23264C4707C8594C18742C6398C14580C20512C6398C9530C19761 用户的搜索词为：短款小背心，请预测用户在电商搜索场景后续行为的语义编码
answer	C1832C15680C24359

Translations are provided solely for reader comprehension:

- **system:** You are a search system. Predict the user’s next action in an e-commerce scenario based on their historical behavior and current query. The behavior sequence is given as semantic IDs, each representing a triplet of attributes, in chronological order.
- **user:** User’s history: [long sequence]. Query: “short cropped vest”. Predict the semantic ID of the next behavior.
- **answer:** C1832C15680C24359

Table 11: Example of the pretrained dataset in the UserAction task for coldstart during deployment. Each sequence is represented as a series of semantic IDs (e.g., C1220C8322C20452).

Field	Content
corpus	C5299C12905C22291C1477C12826C20935C1931C9821C24180C853C14294C22645C2075C12389C16385C853C13340C20935C4894C14014C18136C6398C13220C22786C6398C12675C23264C4707C8594C18742C6398C14580C20512C6398C9530C19761C5727C15620C17409

of the NVIDIA A100 GPU, while featuring a high-capacity memory system with 95.6 GB of on-board memory.

Hyperparameter configurations. Table 12 and 13 summarize the hyperparameters and configuration of models in the SID generation task and generative retrieval task, respectively. For the ID generation task, RQ-VAE employs symmetric 3-layer MLPs (512-64-512) with a 64D discrete codebook and CN-CLIP fusion. The training lasts for 150 epochs using AdamW for stable convergence. For the generative retrieval task, we set the maximum lengths of the historical behaviors to 100, and employ two sequence-to-sequence models in our experiments: a T5-Base model and a Qwen2.5-0.5B model. Both models are fine-tuned on the same dataset with a maximum token length of 1280, where the maximum source and target lengths are set to 1024 and 256, respectively. The T5-Base model is trained for 4 epochs with a per-device batch size of 80, resulting in a total batch size of 1280 using 16 PPU-ZW810E GPUs. It uses a constant learning rate of 2×10^{-4} and the AdamW optimizer with default parameters. In contrast, the Qwen2.5-0.5B-Instruct model is trained for a single epoch with a smaller per-device batch size of 40, also achieving the same total batch size of 1280 using 32 PPU-ZW810E GPUs. It adopts a linear learning rate scheduler with a base learning rate of 5×10^{-5} . Both models are trained using bfloat16 precision to improve training efficiency. The Qwen2.5-3B model follows the same training configuration as the Qwen2.5-0.5B model. All the SIDs are regarded as new tokens and included in the tokenizer of LLMs upon initialization. The dynamic beam size of 3-level and 2-level SIDs during inference is set to {300, 600, 1200} and {600, 1200}, respectively. All codes are freely available at https://github.com/selous123/al_sid.

Table 12: Core configuration of the RQ-VAE model and training setup.

	Parameter	Value
Model	Model Type	RQ-VAE (RQVAE_EMBED_CLIP)
	Codebook Dim	64
	Input Dim	512
	Encoder Structure	3-LayerMLP (512-256-256-64, ReLU)
	Decoder Structure	3-LayerMLP (64-256-256-512, ReLU)
	Fusion Structure	CN-CLIP (chinese-clip-vit-base-patch16)
Training	Epochs	150
	Warmup Epochs	40
	Learning Rate	0.0002
	Optimizer	AdamW (betas=[0.9, 0.999], eps=1e-8)
	LR Scheduler	cosine
	Batch Size	2048

Table 13: Hyperparameters and training configurations of the two models used in Generative Retrieval.

	T5-Base	Qwen2.5-0.5B-Instruct
Seq Length	100	100
Model Type	T5	Qwen2.5
Checkpoint	google-t5/t5-base	Qwen/Qwen2.5-0.5B-Instruct
Max Length	1280	1280
Max Source Length	1024	1024
Max Target Length	256	256
Epochs	4	1
Batch Size (per device)	80	40
Total Train Batch Size	1280	1280
Learning Rate	2e-4	5e-5
LR Scheduler	constant	linear
Optimizer	AdamW (betas=[0.9, 0.999], eps=1e-8)	
bf16	✓	✓
GPU Count	16	32
GPU Type	PPU-ZW810E	PPU-ZW810E
Training Time	20h/epoch	80h/epoch

A.6 Evaluation Metric

This section provides a detailed explanation and calculation of the metrics employed in our paper. We primarily utilize the HitRate to evaluate retrieval performance, while the Embedding HitRate and Gini Coefficient are adopted for the preliminary assessment of SID quality. Additionally, other metrics used for both online and more refined SID evaluation in Figure 7, are also described.

HitRate. It measures whether any of the interacted items during the current page view of each user appear in the retrieval results of recommenders. For the retrieval, all items are ranked according to the predicted scores, and we truncate the list to select the top-K items as the retrieval results. The $HR@K$ can be formulated as

$$HR@K = \frac{1}{N} \sum_{m=1}^N \frac{I_K \& I_{\text{click}}}{I_{\text{click}}}, \quad (5)$$

where N is the total number of test samples, I_{click} represents the set of items actually clicked by the user, and I_K denotes the set of top-K retrieved items.

PVR. Real-world recommender systems typically consist of the retrieval and ranking stages, where there exist multiple retrieval models for online serving. The ranking model accepts thousands of

candidates from these retrieval models and performs more delicate scoring on them. Only dozens of items are ultimately exposed to users as a single page view. PVR computes the ratio of items displayed to users (PV_i) that are successfully retrieved by our retrieval model, which can be formulated as follows:

$$\text{PVR} = \frac{\sum_{mt(i)=FORGE} PV_i}{\sum_I PV_i}, \quad (6)$$

where $mt(i) = FORGE$ indicates that the exposed item i is retrieved by FORGE.

Transaction Count. It is a direct metric that quantifies the number of items successfully purchased by users during a given page view. As such, it serves as one of the most critical indicators of online performance, capturing the real-world impact of the methods in industry. A higher transaction count indicates that the recommended items are not only relevant but also compelling enough to lead to actual purchases.

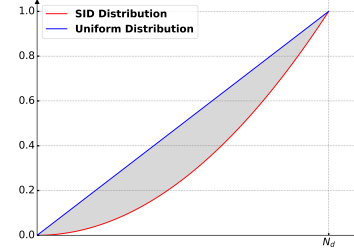
Embedding HitRate. Once we obtain the multi-modal features $\mathcal{H}^i$, they could act as the semantic alternatives of traditional item embeddings. We therefore conduct separate item-to-item retrieval of all historical data based on the similarity among the multi-modal features $\mathcal{H}^i$. The calculation is similar to HitRate except I_K is acquired with mere $\mathcal{H}^i$.

Gini Coefficient. In practice, multiple semantically similar items are mapped to the same SID, while some SIDs remain unused. This uneven distribution leads to inefficiencies in SID usage and introduces unfairness in how identifiers are assigned across the item space. To quantify this imbalance, we adopt the Gini coefficient, which is used to assess the distribution within a population. Given all of the SIDs $\{C_10C_20C_30, \dots, C_18191C_28191C_38191\}$ and the corresponding number of items assigned to each SID $I_c = \{5, 1, \dots, 4\} \in \mathbb{N}^{+N_d}$, we rank the SIDs in increasing order of I_c to obtain a sorted list $S_{id} = \{SID_1, \dots, SID_{N_d}\}$, where N_d is the total number of SIDs. The calculation process for the Gini coefficient is defined as:

$$G = \frac{2}{N_d} \sum_{i=1}^{N_d} \left(\frac{i}{N_d} - L(i) \right), \quad (7)$$

$$L(i) = \frac{C(i)}{C(N_d)}, \quad (8)$$

$$C(i) = \sum_0^i I_c(S_{id}^i). \quad (9)$$



$I_c(S_{id}^i)$ is the associated item count of i -th SID in S_{id} . This can be regarded as the difference of the cumulative distribution function (CDF) between the uniform and the real SID distribution in Figure 5.

Feature Fidelity. It is specifically designed to preserve the original semantic information and avoid its complete loss in the encoding process, like Equation 3. A higher feature fidelity indicates that the multimodal knowledge is better preserved during quantization.

$$\text{Feature Fidelity} = \max \left(0, 1 - \frac{\|\mathcal{H}^i - \mathcal{D}(r_1^{c_1} + \dots + r_m^{c_m})\|_2}{\|\mathcal{H}^i\|_2} \right) \times 100\%. \quad (10)$$

Style/Homogeneous Consistency. In e-commerce scenarios, there are often multiple items in different shops that are either identical (style) or originate from the same source (*homogeneous*, as exemplified in Figure 6). The former refers to two items that are totally identical in size, category, and other attributes. The latter describes two items that are derived from each other through data augmentation techniques such as image editing or text modification. These two metrics evaluate whether items identified as the same style or same origin are assigned to the same SID:

$$\text{Style/Homogeneous} = \frac{\text{Item pairs with the same style/origin in the same SID}}{\text{Total number of item pairs with the same style/origin}} \quad (11)$$



Figure 6: Example of item pairs with the same origin (Homogeneous).

Codebook Utilization. While the Gini Coefficient offers a high-level view of how evenly items are distributed across semantic identifiers (SIDs), it does not directly reflect the actual usage of the available codebook space. We thus provide another metric, codebook utilization, which measures the proportion of codebooks that are actively used to represent items.

$$\text{Utilization} = \frac{|\text{Non-empty codebooks}|}{|\text{Total codebooks}|} \quad (12)$$

A.7 ADDITIONAL Experimental Results

In this section, we conduct more ablation studies on the different configurations of SID optimization to provide a clearer understanding of our methods for researchers.

SID Collision. Table 14 clarifies the impact of different ID collision strategies on the retrieval models. Compared to the base method, which employs a KNN-based strategy to limit the number of items per SID to fewer than 25, the noco variant without any post-processing shows significantly reduced effectiveness. Furthermore, results also indicate that more effective collision avoidance (noco→base→KNN-10→KNN-5) contributes to higher semantic encoding fairness and leads to higher metrics.

To further validate this relationship, we conduct an inverse experiment using the merge method, which intentionally degrades the SID fairness by merging adjacent SIDs below a certain threshold. The substantial drop in performance compared to base confirms a strong positive correlation between codebook utilization and retrieval accuracy.

Distinguished from the KNN-based policy, Random-5 leverages a random-based approach to assign the last level codeword sequentially and randomly. While this reduces collisions and maintains high codebook utilization and SID fairness, it sacrifices meaningful semantic alignment at the final quantization level. The continuous hit rate gain suggests that utilization and collision are more important than the semantic features, as features in the last level capture limited information in the residual quantization.

Data Modality for SID. The i2i (collaborative items) relationships in recommendation systems can provide additional benefits for SID generation. Both the query2i and samecate (items within the same category) approaches underperform the base method, which utilizes only limited i2i data for alignment as in Equation 2. Besides, Table 15 demonstrates that incorporating item-side information (such as seller, price, and category) along with richer i2i relations could lead to more meaningful SIDs and improved retrieval performance.

SID Level Structures. In previous experiments, we primarily considered SID level structures such as 3×8192 (8192_8192_8192) or 2×32768 (32768_32768_32768). This naturally raises the question of whether alternative level structures may offer improved performance in generative retrieval tasks, such as those with more codewords (e.g., 4×4096) or distinct hierarchical setups (e.g., 2048_4096_8192). Extensive validation presented in Table 16 indicates that 3-level SIDs still represent the optimal trade-off between computational efficiency and retrieval effectiveness. In contrast, 2-level structures exhibit significantly worse performance, while 4-level structures incur higher inference costs with negligible gains. Notably, the 1024_4096_32768 configuration achieves more than a 10% improvement in HR@1000 compared to base.

Table 14: Relative improvements of different ID collision alleviating strategies over the base model (3×8192). Improvements are computed as $(M - \text{base})/\text{base}$.

Stage	Method	HR@20		HR@100		HR@500		HR@1000	
S1	base	3.61%	+0.00%	9.67%	+0.00%	20.82%	+0.00%	25.09%	+0.00%
	noco	2.96%	-18.01%	7.69%	-20.48%	17.63%	-15.32%	22.27%	-11.24%
	merge	3.00%	-16.90%	8.32%	-13.96%	18.66%	-10.37%	24.22%	-3.47%
	+KNN-10	3.91%	+8.31%	10.32%	+6.72%	21.85%	+4.95%	25.38%	+1.16%
	+KNN-5	4.18%	+15.79%	10.81%	+11.79%	22.14%	+6.34%	25.30%	+0.84%
	+Random-5	4.67%	+29.36%	11.79%	+21.92%	23.65%	+13.59%	26.30%	+4.82%
S2	base	4.15%	+0.00%	10.70%	+0.00%	22.71%	+0.00%	26.70%	+0.00%
	noco	3.49%	-15.90%	8.91%	-16.73%	19.47%	-14.27%	24.07%	-9.85%
	merge	3.54%	-14.70%	9.11%	-14.86%	20.34%	-10.44%	26.06%	-2.40%
	+KNN-10	4.63%	+11.57%	11.52%	+7.66%	23.24%	+2.33%	26.80%	+0.37%
	+KNN-5	4.81%	+15.90%	11.90%	+11.21%	24.18%	+6.47%	27.09%	+1.46%
	+Random-5	5.23%	+26.02%	12.88%	+20.37%	25.05%	+10.30%	27.74%	+3.90%
S3	base	4.33%	+0.00%	11.16%	+0.00%	23.26%	+0.00%	27.24%	+0.00%
	noco	3.59%	-17.09%	9.24%	-17.20%	20.29%	-12.77%	24.84%	-8.81%
	merge	3.56%	-17.78%	9.55%	-14.43%	20.65%	-11.22%	26.86%	-1.40%
	+KNN-10	4.56%	+5.31%	11.78%	+5.56%	24.06%	+3.44%	27.54%	+1.10%
	+KNN-5	5.08%	+17.32%	12.61%	+12.99%	24.66%	+6.02%	27.72%	+1.76%
	+Random-5	5.43%	+25.40%	13.14%	+17.74%	25.39%	+9.16%	28.03%	+2.90%

Table 15: Relative improvements of different data modalities over the base model (3×8192). Improvements are computed as $(M - \text{base})/\text{base}$.

Stage	Method	HR@20		HR@100		HR@500		HR@1000	
S1	base	3.61%	+0.00%	9.67%	+0.00%	20.82%	+0.00%	25.09%	+0.00%
	query2i	3.20%	-11.36%	8.70%	-10.03%	19.01%	-8.69%	23.93%	-4.62%
	samecate	3.44%	-4.71%	8.92%	-7.76%	19.43%	-6.68%	23.94%	-4.58%
	+sideinfo	3.81%	+5.54%	10.00%	+3.41%	21.69%	+4.18%	26.88%	+7.13%
	+i2i	3.79%	+4.99%	10.04%	+3.83%	21.71%	+4.27%	26.97%	+7.49%
S2	base	4.15%	+0.00%	10.70%	+0.00%	22.71%	+0.00%	26.70%	+0.00%
	query2i	3.71%	-10.60%	9.69%	-9.44%	20.67%	-8.98%	25.72%	-3.67%
	samecate	3.81%	-8.19%	9.86%	-7.85%	21.31%	-6.16%	25.78%	-3.45%
	+sideinfo	4.32%	+4.10%	11.06%	+3.36%	23.46%	+3.30%	28.43%	+6.48%
	+i2i	4.12%	-0.72%	10.69%	-0.09%	23.06%	+1.54%	28.10%	+5.24%
S3	base	4.33%	+0.00%	11.16%	+0.00%	23.26%	+0.00%	27.24%	+0.00%
	query2i	3.67%	-15.24%	9.99%	-10.48%	21.77%	-6.41%	27.05%	-0.70%
	samecate	3.98%	-8.08%	10.23%	-8.33%	21.91%	-5.80%	26.50%	-2.72%
	+sideinfo	4.34%	+0.23%	11.43%	+2.42%	24.20%	+4.04%	29.32%	+7.64%
	+i2i	4.22%	-2.54%	11.32%	+1.43%	23.69%	+1.85%	28.91%	+6.13%

SID Quantization Methods. The schema illustrated in Figure 2(b) outlines the quantization process using RQ-VAE in FORGE. As other related works [24, 46, 47] have explored alternative encoding methods such as RQ-Kmeans, we conduct a comparative analysis and present the results in Table 17. First, both the semantic-related and residual decoding strategies play a critical role in retrieval performance. The Random baseline assigns SIDs to items without considering any multimodal features, leading to a significant 70% drop in HR@1000. In contrast, the Multiple-VQ approach employs separate encoders at each level, resulting in decoupled codewords within SIDs and consequently suboptimal representation learning. Second, RQ-Kmeans achieves performance

Table 16: Relative improvements of different SID level structures over the base model (3×8192). Improvements are computed as $(M - \text{base})/\text{base}$.

Stage	Method	HR@20		HR@100		HR@500		HR@1000	
S1	base	3.61%	+0.00%	9.67%	+0.00%	20.82%	+0.00%	25.09%	+0.00%
	2048_4096_8192	3.20%	-11.36%	8.70%	-10.03%	19.01%	-8.69%	23.93%	-4.62%
	8192_4096_2048	3.44%	-4.71%	8.92%	-7.76%	19.43%	-6.68%	23.94%	-4.58%
	2x32768	3.37%	-6.65%	9.09%	-6.00%	20.33%	-2.35%	26.94%	+7.37%
	3x512	2.84%	-21.33%	7.80%	-19.34%	17.63%	-15.32%	23.33%	-7.01%
	1024_4096_32768	3.76%	+4.16%	10.09%	+4.34%	22.03%	+5.81%	27.95%	+11.40%
	4x4096	3.65%	+1.11%	9.50%	-1.76%	20.25%	-2.74%	25.06%	-0.12%
S2	base	4.15%	+0.00%	10.70%	+0.00%	22.71%	+0.00%	26.70%	+0.00%
	2048_4096_8192	3.96%	-4.58%	10.44%	-2.43%	22.17%	-2.38%	27.64%	+3.52%
	8192_4096_2048	4.02%	-3.13%	10.52%	-1.68%	22.54%	-0.75%	28.21%	+5.66%
	2x32768	3.85%	-7.23%	10.03%	-6.26%	21.93%	-3.43%	28.96%	+8.46%
	3x512	3.34%	-19.52%	8.65%	-19.16%	19.46%	-14.31%	24.89%	-6.78%
	1024_4096_32768	4.34%	+4.58%	11.07%	+3.46%	23.58%	+3.83%	29.75%	+11.42%
	4x4096	4.19%	+0.96%	10.53%	-1.59%	21.49%	-5.37%	26.11%	-2.21%
S3	base	4.33%	+0.00%	11.16%	+0.00%	23.26%	+0.00%	27.24%	+0.00%
	2048_4096_8192	4.04%	-6.70%	10.50%	-5.91%	23.15%	-0.47%	29.07%	+6.72%
	8192_4096_2048	4.19%	-3.23%	10.92%	-2.15%	23.59%	+1.42%	28.99%	+6.42%
	2x32768	3.91%	-9.70%	10.49%	-6.00%	22.65%	-2.62%	30.00%	+10.13%
	3x512	3.49%	-19.40%	9.07%	-18.73%	19.99%	-14.06%	25.96%	-4.70%
	1024_4096_32768	4.30%	-0.69%	11.17%	+0.09%	24.14%	+3.78%	30.35%	+11.42%
	4x4096	4.00%	-7.62%	10.46%	-6.27%	21.72%	-6.62%	26.45%	-2.90%

comparable to RQ-VAE (base) on our datasets. As noted by OneRec [24], a key advantage of RQ-Kmeans lies in the utilization of codebooks. However, with post-processing to handle ID collisions, base and ours implemented with RQ-VAE demonstrate a clear superiority over RQ-Kmeans.

Additional Evaluation Metrics. We further calculate extensive metrics described in Section A.6. **i)** As we proposed in previous content, the embedding hitrate, fairness, and utilization are examined to be positive with retrieval performance. The consistent ranking Ours > base > RQ-Kmeans > Multiple-VQ observed in both Figure 7 and Table 17 also validates this conclusion. **ii)** Other metrics like feature fidelity do not show a clear correlation with retrieval effectiveness. Multiple-VQ achieves the best scores on two of them, but in Table 17 we can observe that it obtains the worst retrieval results. A similar pattern is observed for RQ-Kmeans, which significantly outperforms base in feature fidelity but still lags in overall retrieval performance.

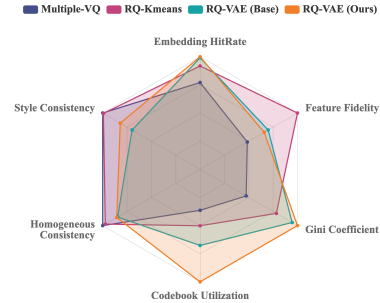


Figure 7: Performance of quantizations under different metrics.

A.8 Optimized lm_head

To reduce the computational overhead during training, we only compute the necessary logits starting from the first valid label position in each sequence. This is achieved in Algorithm 3 by identifying the first non-negative label index using the `min_first_non_neg_index` function, and slicing the hidden states accordingly before applying the language model head. Figure 8 clearly shows that the optimization significantly reduces both memory usage and computation time.

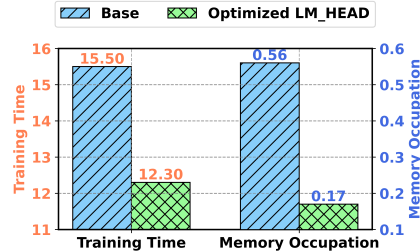


Figure 8: Efficiency gain with optimized LM_HEAD.

Table 17: Comparison of different quantization methods across stages.

Stage	Method	HR@20	HR@100	HR@500	HR@1000
S1	Random	1.08%	2.91%	4.82%	4.82%
	Multiple-VQ	2.87%	8.15%	18.30%	23.84%
	RQ-Kmeans	3.42%	9.39%	20.71%	<u>26.11%</u>
	RQ-VAE (base)	<u>3.61%</u>	<u>9.67%</u>	<u>20.82%</u>	25.09%
	RQ-VAE (Ours)	4.89%	12.31%	24.79%	29.01%
S2	Random	1.66%	4.09%	6.66%	6.66%
	Multiple-VQ	3.33%	8.68%	19.55%	25.36%
	RQ-Kmeans	3.79%	10.09%	22.23%	<u>27.95%</u>
	RQ-VAE (base)	<u>4.15%</u>	<u>10.70%</u>	<u>22.71%</u>	26.70%
	RQ-VAE (Ours)	5.33%	13.23%	26.37%	30.28%
S3	Random	1.51%	4.04%	6.41%	6.41%
	VQ	3.47%	9.28%	20.69%	26.47%
	RQ-Kmeans	3.96%	10.82%	23.00%	<u>28.71%</u>
	RQ-VAE (base)	<u>4.33%</u>	<u>11.16%</u>	<u>23.26%</u>	27.24%
	RQ-VAE (Ours)	5.44%	13.79%	26.71%	30.86%

Algorithm 3 Efficient Logits Computation during Training

```

1: function FORWARD(input_ids, labels)
2:   outputs  $\leftarrow$  BASE_MODEL(input_ids)
3:   hidden_states  $\leftarrow$  outputs.last_hidden_state
4:   if training then
5:     first_non_neg  $\leftarrow$  MIN_FIRST_NON_NEG_INDEX(labels)
6:     logits_to_keep  $\leftarrow$  labels.size(1) - first_non_neg + 1
7:     labels  $\leftarrow$  labels[:, -logits_to_keep :]
8:   end if
9:   slice_indices  $\leftarrow$  SLICE(-logits_to_keep, None)
10:  logits  $\leftarrow$  LM_HEAD(hidden_states[:, slice_indices, :])
11:  loss  $\leftarrow$  CROSS_ENTROPY(logits, labels)
12:  return loss, logits, outputs
13: end function
14: function MIN_FIRST_NON_NEG_INDEX(labels)
15:  mask  $\leftarrow$  (labels  $\geq$  0).cumsum(dim = -1)
16:  first_non_neg  $\leftarrow$  ARGMAX((mask == 1).float(), dim = -1)
17:  return min(first_non_neg)
18: end function

```

A.9 Case Study

Figure 9 offers an intuitive insight into the hierarchical structure of semantic identifiers generated after optimization under RQ-VAE. **i)** The semantic information on which the generation process relies ensures that items sharing the same SID have similar or identical meanings. **ii)** The residual-based computation results in a weaker semantic distinction at the last level of the hierarchical codebook, where products mapped to different third-level codebooks (e.g., $C_14C_24328C_3859$ and $C_14C_24328C_31623$) show semantic consistency. **iii)** Codebooks at earlier levels have a greater impact on semantics. Changing the 2-level codebook shifts the item category to clothing, while remaining within the drawer class. However, modifying the first-level codebook directly switches the item to an electronic device. These changes are consistent with our multimodal feature-based residual quantization generation.



Figure 9: Visualization of corresponding items under modification to the SID.

A.10 Notations

All the notations used in this paper can be found in Table 18.

Table 18: Summary of Notations.

Symbol	Description
Item Features	
$\mathcal{I}_{\text{text}}^i$	Text description of item i
$\mathcal{I}_{\text{image}}^i$	Image of item i
i^+	The item that owns the most collaborative relations with item i
i^-	Items collected with in-batch negative sampling
Feature Extraction	
$\mathcal{M}_{\text{text}}$	Text Encoder
$\mathcal{M}_{\text{image}}$	Image Encoder
$\mathcal{M}_{\text{Fusion}}$	Model used to Aggregate Multi-modal Features
$\mathcal{H}^i$	Extracted multi-modal features of item i
Item Tokenization	
$\mathcal{E}$	Encoder of SID generation
m	Total level of the SIDs
$z_1, \dots, z_m$	Input for codebooks of each level
$c_1, \dots, c_m$	Identifier/Codebook of each level
$\mathcal{D}$	Decoder for reconstruction
Loss Function	
$\mathcal{L}_{\text{cont}}$	Contrastive loss for alignment between relative items
$\mathcal{L}_{\text{recon}}$	Reconstruction loss to retain semantic information
$\mathcal{L}_{\text{rec}}$	The next-token prediction loss for retrieval